

AI-POWERED HEALTHCARE AGE PREDICTION USING SALESFORCE AND ML

USE CASE:

Accurately predicting an individual's age based on various data inputs is a challenging task due to the inherent variability and complexity of the data. Traditional methods of age verification can be intrusive, time-consuming, and prone to errors. Inadequate age prediction can lead to non-compliance with regulations, suboptimal personalization, and security vulnerabilities. Age prediction involves estimating an individual's age using various data inputs, such as images, text, voice, or other biometric information.

TEAM MEMBERS :-

ARYAMAN - 22911A3503

CHANDRA SHEKHAR - 22911A3524

ABHILASH REDDY - 22911A3563

BOLLAJU KEERTHI - 22911A3572

DASARI PRANAVA - 22911A3575

KAVALI SNEHITHA - 22911A3590

INTRODUCTION

Healthcare systems depend on accurate and reliable patient information to ensure effective treatment and service delivery. Among all attributes, age is one of the most critical factors in medical decision-making. It directly influences:

Treatment Personalization – Doctors adjust prescriptions, dosages, and therapies based on patient age.

Clinical Trial Eligibility – Research studies often require participants within specific age brackets for valid results.

Insurance and Compliance Checks – Age impacts insurance eligibility, premium calculations, and adherence to regulatory standards.

However, **traditional methods of collecting and verifying age information are often manual, slow, and prone to human error**. This can cause inefficiencies in healthcare workflows and may even affect the quality of care provided to patients.

To address this challenge, our project integrates **Artificial Intelligence (AI)** with **Salesforce** to deliver a fast, reliable, and automated solution for **age detection**. By leveraging deep learning models trained on image datasets, the system estimates age from facial images and seamlessly stores predictions in Salesforce. This integration ensures that healthcare professionals can quickly access accurate patient information within their existing workflows.

Beyond healthcare, this solution has the potential to extend to other industries such as **retail, security, forensics, and social media**, where age prediction plays an important role in personalization, safety, and compliance.

OBJECTIVE

The objective of this project is to build an AI-powered system in Salesforce to predict patient age quickly, securely, and accurately. The solution integrates a machine learning model with Salesforce, enabling healthcare professionals to obtain automated predictions directly from patient photos within the Salesforce environment.

Key Goals

- Automate **age prediction** from patient photos and details
- Integrate the solution with **Salesforce Lightning Web Components (LWC)** and **custom**

objects

- Deploy the **machine learning model** as a REST API and connect via **Apex callouts**
- Provide doctors with **secure access** to predictions directly in Salesforce
- Display prediction results on the **Lightning Record Page** for easy viewing and reporting

APPROCHES

APPROACH 1:

Step 1: Image Upload via Lightning Web Component (LWC)

The process begins when the user uploads a patient image through a custom **Lightning Web Component (LWC)**.

- The LWC performs **basic client-side validations** such as file type and size before accepting the upload.
- Once validated, the image is prepared for processing and sent to Salesforce backend.

Step 2: Apex Trigger and API Callout

An **Apex Trigger** is configured to capture the upload event on the custom object.

- The trigger invokes an **Apex class** that encodes the image into **Base64 format**.
- This encoded image is sent to the deployed **Flask API endpoint** which is registered through Named Credentials via an Apex callout for further processing.

Step 3: Model Processing and Prediction

The Flask API hosts a **VGG16 Convolutional Neural Network (CNN)** model designed to predict **age and gender** from facial images.

- The API first applies **preprocessing steps** such as face detection, resizing, and normalization.
- The VGG16 model then performs inference and returns the predictions with an accuracy of approximately **81 %**.

Step 4: Displaying Predicted Output in Salesforce

The API response is parsed by the Apex class, and the predicted **age and gender** values are updated into the respective Salesforce record fields (Predicted_Age__c and Predicted_Gender__c).

- The predictions are then displayed dynamically in the **Lightning Web Component (LWC)** on the **Lightning Record Page**, allowing users (e.g., doctors) to view results seamlessly within Salesforce.

Step 5: Testing and Validation

The complete workflow was validated using **sample images**.

- Verified that the image upload, API callout, model inference, and record updates worked

smoothly end-to-end.

- Confirmed that predictions were accurately reflected in Salesforce fields and displayed correctly through the LWC.

APPROACH 2:

Step 1: Train Model

The machine learning model is trained using a **Kaggle dataset** containing images with age and gender information.

- Implemented using **Python ML libraries** such as TensorFlow/Keras.
- Preprocessing steps included resizing, normalization, and augmentation to improve performance.
- The trained model is exported into a serialized format (e.g., .h5 or .pkl) for deployment.

Step 2: Deploy Model

The trained model is hosted on a **cloud platform** such as AWS, GCP, or Heroku.

- Wrapped with a **Flask API** that exposes a /predict endpoint.
- Predictions are returned in JSON format (e.g., { "age": 28, "confidence": 0.84 }).

Step 3: Register in Salesforce

The deployed API is integrated into Salesforce using **Einstein Studio (Bring Your Own Model – BYOM)**.

- The API is registered as an **external service** with proper authentication.
- Input parameters (image data) and output fields (predicted age, confidence) are mapped.

Step 4: Create Custom Object

A custom Salesforce object named “**Age Prediction Record**” is created to store prediction details.

- Fields include:
 - Input Data (image or reference)
 - Predicted Age
 - Confidence Score
 - Timestamp

Step 5: Automate with Flow

Salesforce **Flows** are configured to automate prediction requests.

- When a new record is created, the flow triggers an **Apex callout** to the external API.
- The API response is parsed, and record fields are updated automatically.

Step 6: Build LWC UI

A **Lightning Web Component (LWC)** is developed to provide a user-friendly interface.

- Allows users to **upload input (image or text)** directly.

- Creates a corresponding record in Salesforce.
- Displays predicted results in real-time.

Step 7: Add to Lightning Page

The LWC is placed into a **Lightning App Builder** page layout.

- Provides doctors and healthcare staff easy access to predictions directly within Salesforce.

Step 8: Test End-to-End

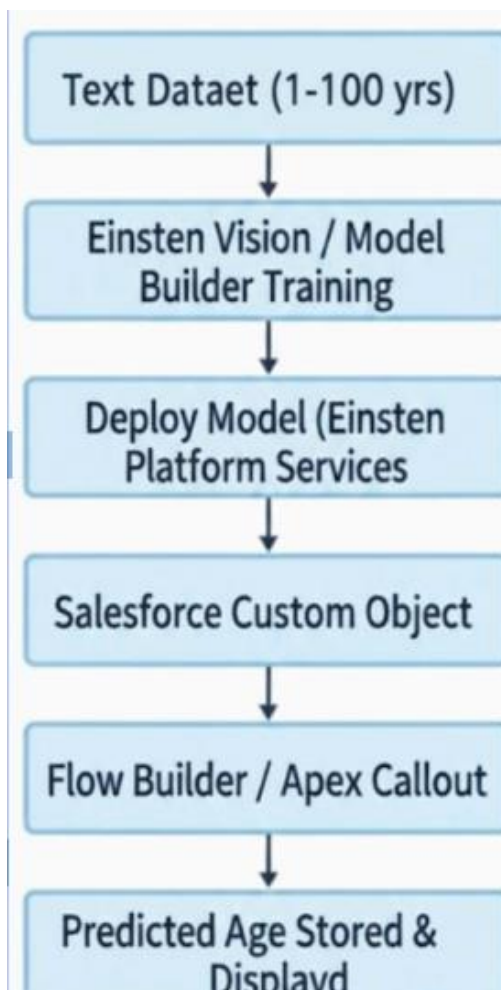
The workflow is tested with **sample data**.

- Verified that predictions are successfully generated when an image is uploaded.
- Confirmed that records are created, updated, and displayed accurately in the LWC.

Step 9: Monitor & Maintain

Ongoing monitoring is established for system performance and reliability.

- Track API usage and performance metrics.
- Update the machine learning model periodically with new data.
- Provide **analytics and dashboards** within Salesforce to track prediction history and accuracy.



APPROACH 3:

Step 1: Collect Image Dataset

A dataset of labeled facial images is collected, covering individuals aged **1–100 years**.

- Images are stored in **JPG or PNG** format.
- Each image is tagged with the correct **age label** for supervised training.

Step 2: Upload Dataset (Einstein Model Builder)

The dataset is uploaded into **Einstein Vision / Model Builder** within Salesforce.

- This serves as the training source for the custom model.

Step 3: Configure Model (Einstein Model Builder)

The model is configured with appropriate training parameters:

- Image size
- Data augmentation settings
- Number of epochs
- Batch size and learning rate

Step 4: Train Model (Einstein Model Builder)

Einstein Vision automatically trains the model using the uploaded dataset.

- The model learns to recognize facial patterns and predict **age** based on image features.

Step 5: Deploy Model (Einstein Platform Services)

The trained model is deployed as a **REST API endpoint** through Einstein Platform Services.

- The endpoint accepts input images and returns predictions in **JSON format** containing age and confidence scores.

Step 6: Upload Input (LWC / Record Page)

Users upload patient images through:

- A **Lightning Web Component (LWC)**
- Or directly on a **Salesforce Record Page**
- Supported formats include **JPG and PNG**

Step 7: Integration (Flow Builder / Apex Callout)

The uploaded images are sent to the deployed Einstein API:

- **Salesforce Flow or Apex callouts** handle the request.
- The API responds with **predicted age and confidence score** in JSON format.

Step 8: Store Prediction (Salesforce Custom Object)

Predictions are stored in a dedicated **Salesforce Custom Object** for tracking.

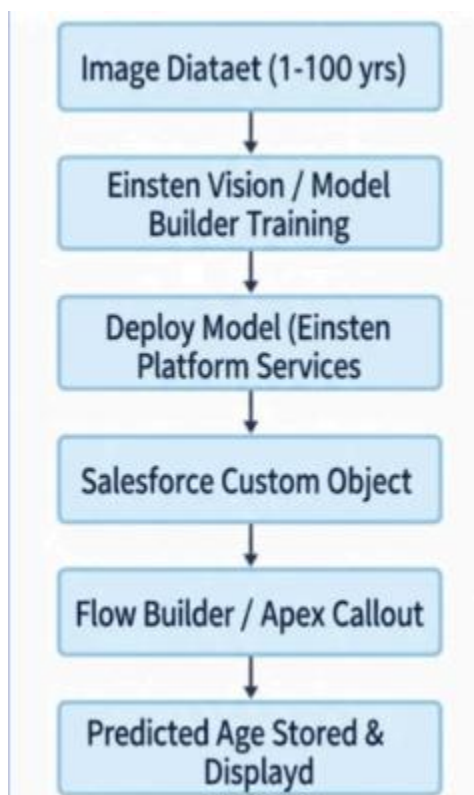
- Fields include:

- Predicted Age
- Confidence Score
- Timestamp

Step 9: Display & Insights (Reports / Dashboards / Automation)

The stored predictions are used across Salesforce features:

- Displayed on **record pages** via LWC or standard components
- Summarized in **reports and dashboards** for insights
- Integrated into **automation workflows** for notifications and further actions



PROPOSED SOLUTION

Step 1: Image Upload via LWC

Users upload images through a custom Lightning Web Component (LWC).

- The component provides instant feedback and displays a placeholder/loading indicator while the image is being processed.
- Performs client-side validation such as checking file type and file size before submission.
- Once validated, the image is sent to the Salesforce backend (Apex) for further processing through an API callout.

Step 2: API Callout & Apex

- API url is registered into Salesforce using Named Credentials
- The trigger invokes an Apex class that prepares and sends the image to the Flask API hosted at: <https://salesforce-age-genderapi.onrender.com/predict>
- The image is Base64-encoded and sent as a JSON payload through an HTTP callout.
- The Apex class receives and parses the API response containing predicted age and gender.

Step 3: Salesforce Record Update

The parsed API response is used to update the respective Salesforce fields:

- Predicted_Age__c
- Predicted_Gender__c

These updated fields become available for use in workflows, reports, and dashboards, ensuring that predictions are seamlessly integrated into Salesforce processes.

Step 4: Model & API (Supportive Backend)

The predictive backend is powered by a VGG16 Convolutional Neural Network (CNN) model.

- Deployed as a Flask API on Render (note: running heavy models requires a paid instance).
- Includes preprocessing steps such as face detection, image resizing, and normalization to improve model performance.
- Achieves approximately 81% accuracy in predicting age and gender.

Step 5: End-to-End Integration Workflow

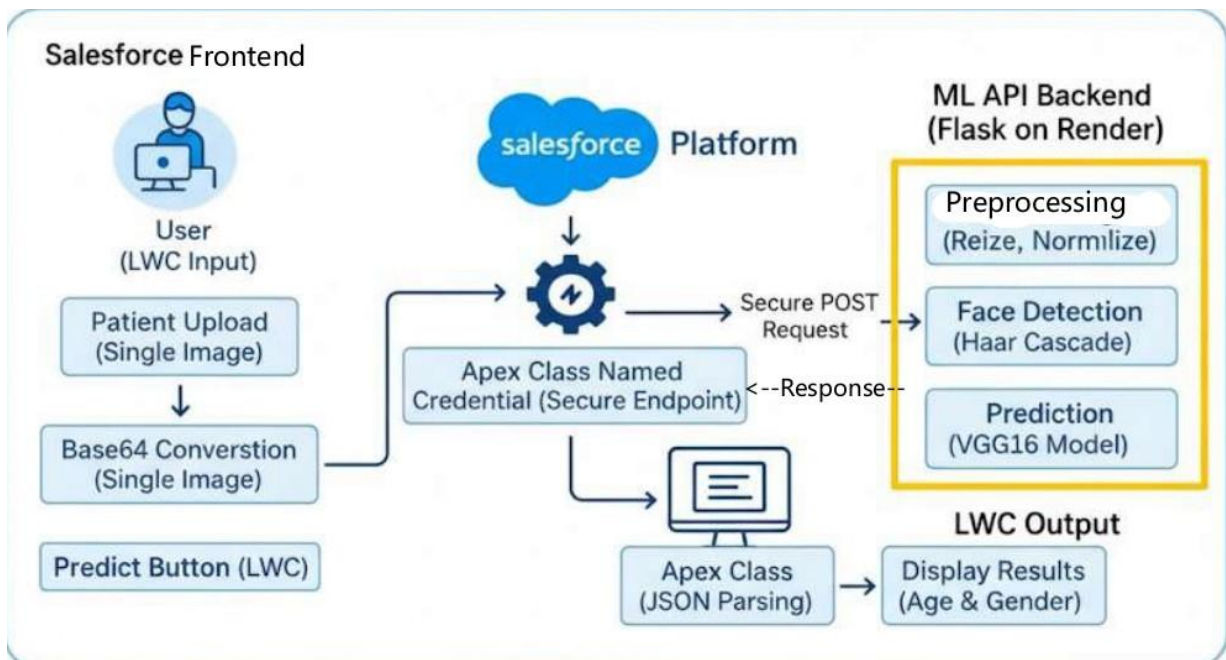
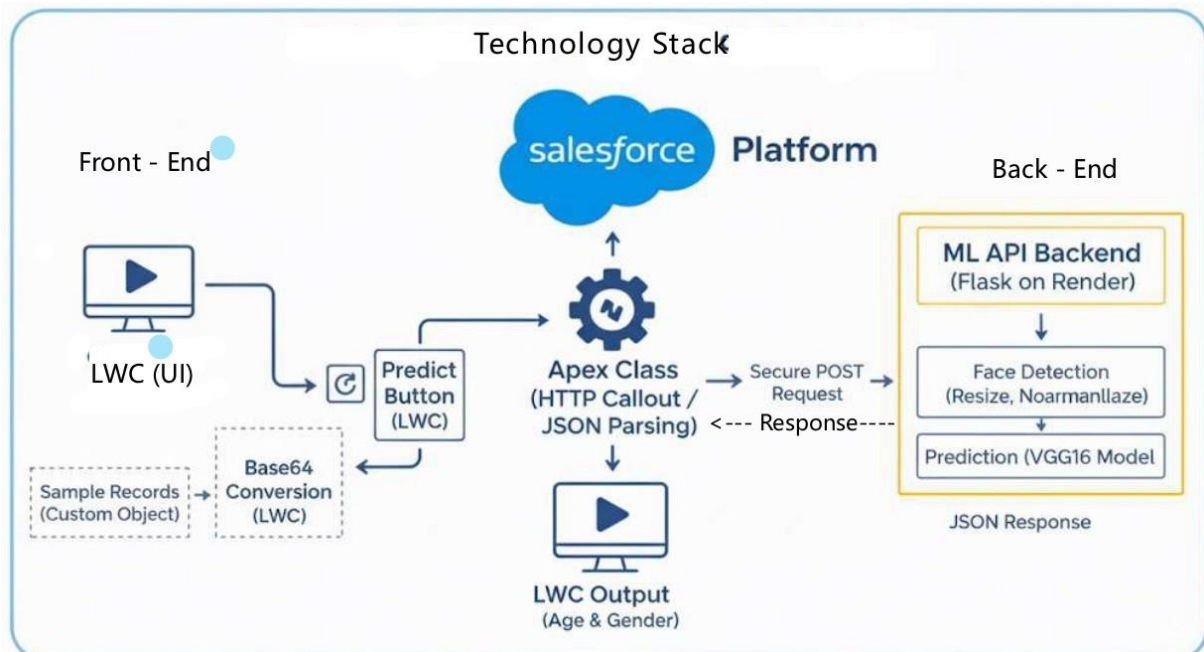
The overall integration flow is as follows:

1. User uploads an image via LWC
2. Apex Trigger captures the upload event
3. Apex class performs API callout to Flask backend
4. Model processes the image and returns predictions
5. Salesforce record fields (Predicted_Age__c, Predicted_Gender__c) are automatically updated

This creates a seamless AI-powered prediction pipeline within Salesforce.

Step 6: Testing & Validation

- Conducted API callout tests using Apex and sample images.
- Verified that the API response correctly mapped to Salesforce fields.
- Demonstrated the end-to-end workflow conceptually, noting that full-scale production execution requires a paid Render instance for hosting the model.



FRONTEND

A custom Salesforce app was created to serve as the interface for the age prediction feature.

- Within this app, Lightning Web Component (LWC) templates were developed to display the predicted age in a clear and intuitive manner.
- The predicted age is displayed dynamically on the templates, providing real-time feedback to the user.
- The design ensures a seamless and user-friendly experience, allowing doctors or staff to view predictions easily.
- The setup creates an interactive and visually organized interface, enabling efficient management of patient records alongside AI-generated predictions.

FRONTEND WORKFLOW

1. App Creation

A custom Salesforce app was created to serve as the primary interface for the age prediction feature.

- The app acts as a central workspace, providing users access to all functionalities related to age prediction.

2. Building Templates

Within the app, Lightning Web Component (LWC) templates were developed.

- These templates display predicted age in a clear and interactive manner.
- Designed to ensure a user-friendly layout for both input and output data, making interactions intuitive.

3. Doctor Input and Interaction

Doctors can enter patient details or upload images directly through the app interface.

- The frontend captures these inputs and triggers the backend process via API callouts for prediction.

4. Displaying Results

Once the backend returns predictions, the LWC templates dynamically display the results on the record page.

- Additional information, such as confidence scores and timestamps, can also be shown to provide better insights.

5. Seamless User Experience

The frontend ensures smooth interaction between the user and the system.

- Integration of forms, templates, and dynamic display components provides a visually organized and efficient workflow.
- Users can easily view age predictions and manage patient records without switching between multiple interfaces.

This screenshot shows the home page of the 'Age Predictor' application. The header includes a search bar and navigation links for Home, Dashboards, Reports, Contacts, Age Prediction Records, and Patients. The main content area features a large blue background with a white card in the center that says 'WELCOME TO Salesforce Age Prediction Service' and an 'Enter' button. Below this, there is a 'Quarterly Performance' section with a line chart showing 'CLOSED' and 'OPEN (>70%)' metrics, and a 'GOAL' section. The chart shows a steady increase in performance over time, with the 'CLOSED' line reaching 500k and the 'OPEN (>70%)' line reaching 400k. The 'GOAL' section shows a target of 300k.

This screenshot shows the 'Patient Details' form within the 'Age Predictor' application. The header is identical to the home page. The main content area features a white card with the title 'Patient Details'. Below the title, there is a text input field containing 'John', an 'Upload Image' button, and a 'Choose file' button with a file name '5.jpg'. At the bottom of the card, there is a blue 'Predict Age & Gender' button and a grey 'Back' button. Below the card, there is a 'Quarterly Performance' section with a line chart showing 'CLOSED' and 'OPEN (>70%)' metrics, and a 'GOAL' section. The chart shows a steady increase in performance over time, with the 'CLOSED' line reaching 500k and the 'OPEN (>70%)' line reaching 400k. The 'GOAL' section shows a target of 300k.

BACKEND

The backend is powered by a Flask/FastAPI API that handles image processing and prediction:

- The API receives images uploaded from Salesforce, either in Base64 or multipart format.
- Preprocessing steps are applied to prepare the image for the model:
 - Converted into a numerical array
 - Resized to match the model's input dimensions
 - Normalized to standardize pixel values for accurate predictions
- The preprocessed image is passed to a VGG16 Convolutional Neural Network (CNN) model.
- The model predicts age and gender, and the results are formatted as a JSON response, for example:
- { "predicted_age": 25, "predicted_gender": "Male" }
- This response is sent back to Salesforce, ensuring the frontend receives a ready-to-use prediction seamlessly and dynamically.

BACKEND WORKFLOW

1. Data Collection & Preprocessing

- Collected healthcare and facial image datasets containing age and gender labels.
- Cleaned and preprocessed the dataset to ensure high-quality inputs for model training.
- Used the UTKFace Dataset consisting of 20,000 images, with ages ranging from 0 to 116 years.

2. Model Training

- Trained a VGG16 Convolutional Neural Network (CNN) model using Python ML libraries in Google Colab or VS Code.
- Applied transfer learning to improve training efficiency.
- Image preprocessing was performed through OpenCV, including resizing, normalization, and augmentation.
- The model learned to predict both age and gender from facial images.

3. Deploy Model via Render

- The trained model was uploaded to Render and deployed as a REST API endpoint (/predict).
- The API accepts image inputs and returns predictions in JSON format, for example:

- { "age": 37, "gender": "Male" }

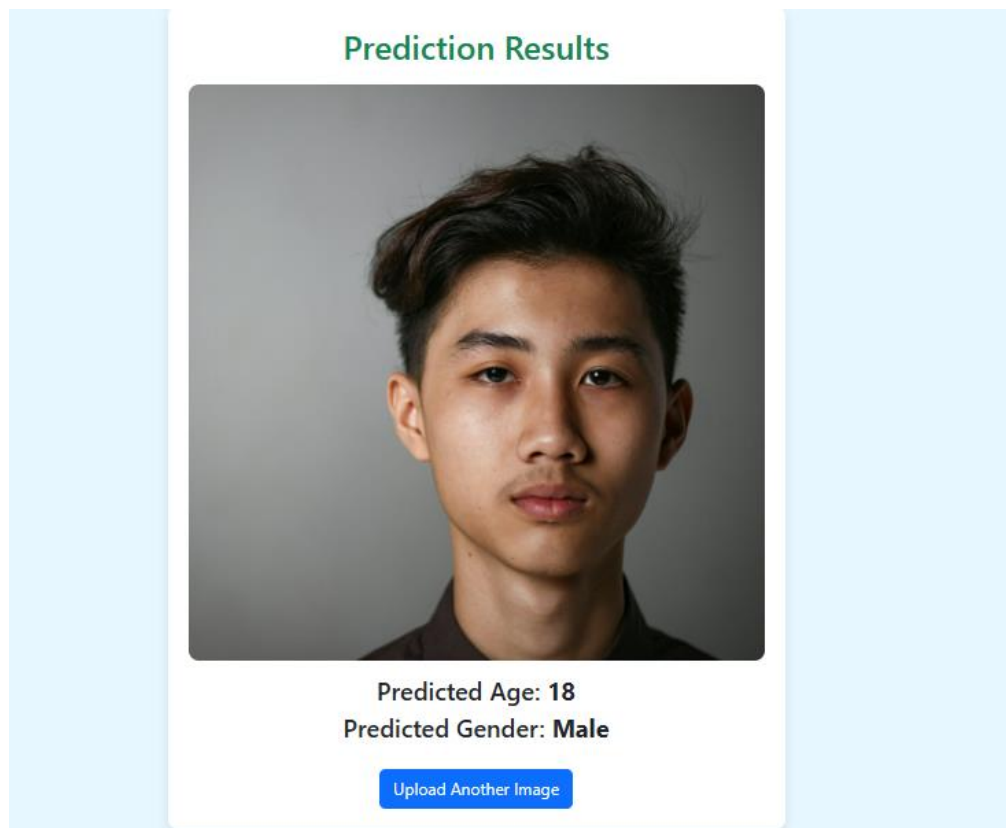
4. Salesforce Integration

- Configured Named Credentials in Salesforce to authenticate API requests.
- Registered the Render API as an External Service.
- Built a Flow and Lightning Web Component (LWC) that sends patient images or data to the API and retrieves predicted age and gender via Apex callouts.

5. Storing & Displaying Predictions

- The API response is stored in a custom Salesforce object, e.g., *Age Prediction Request*, with fields for:
 - Predicted age
 - Predicted gender
 - Timestamp
- Predictions are displayed dynamically in the frontend Lightning Web Component, providing doctors with a seamless and interactive interface to view results.

OUTPUT:



Prediction Results



Predicted Age: 23
Predicted Gender: Female

[Upload Another Image](#)

Prediction Results



Predicted Age: 75
Predicted Gender: Male

[Upload Another Image](#)

INTEGRATION

Step 1: Backend API

- The machine learning model is hosted on a Flask/FastAPI server.
- Exposes a /predict endpoint that receives images and returns a JSON response containing the predicted age and gender, for example:
- { "predicted_age": 25, "predicted_gender": "Male" }

Step 2: Salesforce Integration

- Named Credentials are configured in Salesforce to store the API URL and authentication details securely, avoiding hardcoding in Apex.
- An Apex class performs the callout to the backend API:
 - Receives the uploaded image from the Lightning Web Component (LWC) in Base64 format.
 - Sends the image to the Flask/FastAPI API.
 - Parses the JSON response and extracts the predicted age and gender.
- Acts as a middleware between the frontend and the backend, ensuring secure and reliable data transfer.

Step 3: Lightning Web Component (Frontend)

- Users upload images through the LWC interface.
- The LWC calls the Apex method, which in turn calls the API and retrieves predictions.
- Once the response is received, the LWC dynamically displays the predicted age and gender to the user, providing a seamless and interactive experience.

MODEL PERFORMANCE

The trained model was evaluated on the test dataset to measure its predictive performance. The key metrics obtained are as follows:

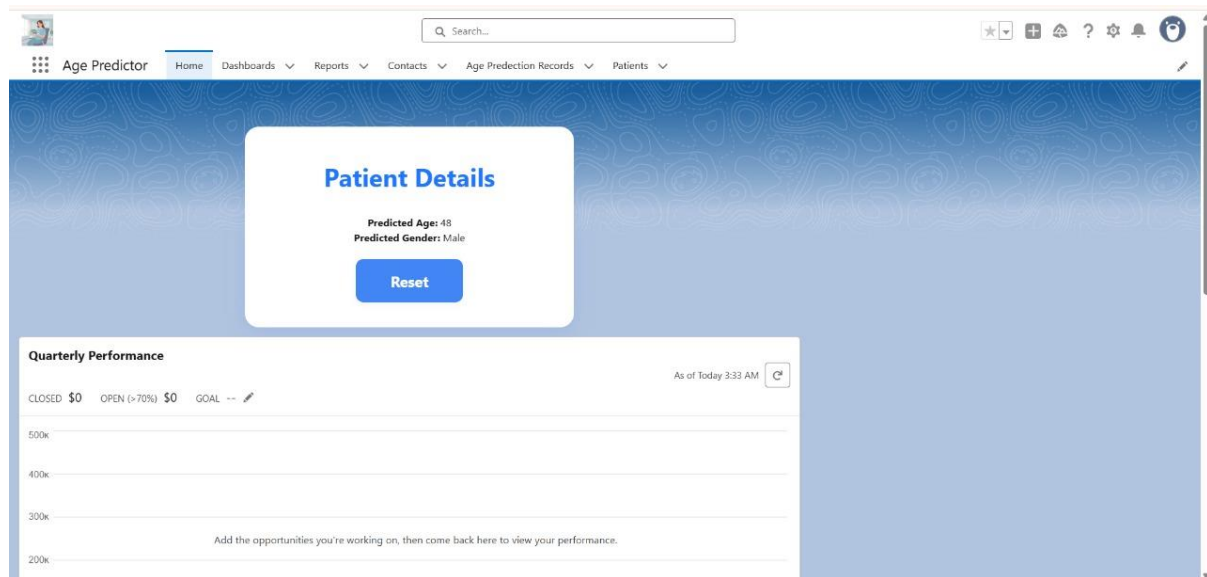
1. Accuracy: 81%
 - Accuracy represents the proportion of total predictions that the model correctly classified.
 - An accuracy of 81% indicates that the model correctly predicts outcomes in 8 out of 10 cases, demonstrating reliable overall performance.
 - While accuracy is a good indicator of general correctness, it may not fully capture performance if the dataset has imbalanced classes.

2. Precision: 85%

- Precision measures how many of the positive predictions made by the model are actually correct.
- With a precision of 85%, the model correctly identifies 85 out of 100 predicted positives, showing that it makes very few false positive errors.
- High precision is important in healthcare applications, as it ensures that the model rarely misclassifies cases, supporting trustworthy predictions.

3. Interpretation

- Together, the accuracy and precision indicate that the model is both generally correct and reliable in its positive predictions.
- This level of performance is suitable for deployment in the age prediction workflow, providing consistent and actionable insights.



LIMITATIONS

1. System Performance

- **Latency Issues** – Image conversion to Base64 and API callouts can introduce delays, especially with large image files or poor network conditions.
- **Scalability Challenges** – The current setup may face difficulties handling high volumes of concurrent requests in production without a more robust cloud deployment (e.g., AWS SageMaker, GCP).

2. Salesforce & API Constraints

- **Einstein Studio Constraints** – Limited flexibility in customization and performance tuning once

the API is registered in Salesforce.

- **Accuracy Challenges** – Predictions may not always be reliable for critical use cases without ongoing retraining and model monitoring.
- **Maintenance Overhead** – Continuous monitoring, model retraining, and API updates are required to maintain optimal performance.

3. Model & Data Limitations

- **Accuracy Concerns** – A single CNN model may not provide consistent accuracy across diverse demographics or varying image qualities.
- **Security & Compliance** – Transferring patient images outside Salesforce may raise **data privacy** concerns and require compliance with regulations such as **HIPAA** or **GDPR**.

FUTURE ENHANCEMENTS

The system can be extended to support multi-modal inputs, including text, voice, and biometric data, which would improve the accuracy of age estimation. Leveraging more advanced deep learning architectures, such as transformers, can further reduce prediction error rates and enhance reliability. Implementing real-time prediction capabilities would enable live applications, including telemedicine consultations and online patient onboarding. Security and compliance can be strengthened through robust encryption, consent management, and adherence to regulations such as GDPR and HIPAA. For large-scale deployment, hosting the model on platforms like AWS SageMaker, GCP Vertex AI, or Azure ML would improve scalability and performance. Additionally, predicted age can be utilized to provide personalized services, targeted healthcare plans, or product recommendations. Finally, building real-time dashboards in Salesforce would allow monitoring of predictions, accuracy trends, and system usage, providing actionable insights for continuous improvement.

APPLICATIONS

- **Healthcare & Wellness:** Age prediction can assist healthcare providers in patient monitoring, early diagnosis, personalized treatment plans, and telemedicine consultations. It can also support wellness programs by tailoring health recommendations according to age groups.
- **Security & Access Control:** Systems can implement age-based verification to restrict entry to physical locations or digital services, such as age-restricted apps, websites, or secure areas in facilities.
- **Retail & Marketing:** Age prediction enables targeted marketing campaigns and personalized product recommendations, enhancing customer engagement and sales effectiveness. Retailers

can optimize promotions based on the demographic profile of their customers.

- **Forensics & Law Enforcement:** Age estimation aids in identifying individuals in criminal investigations, missing person cases, or surveillance analysis. It provides additional information for investigative procedures and profiling.
- **Social Media & Apps:** Age prediction can help platforms deliver age-appropriate content, enforce platform restrictions for minors, and improve user experience by customizing interactions based on estimated age.
- **Human-Computer Interaction (HCI):** Adaptive interfaces can be designed to suit different age groups. For example, UI/UX elements, font sizes, or interaction patterns can be adjusted dynamically based on the predicted age of the user.
- **Automotive & Safety Systems:** Age prediction can be integrated into driver monitoring systems to provide safety alerts or adaptive controls based on the driver's age, supporting safer driving and accident prevention.
- **Fitness & Lifestyle:** Fitness and lifestyle applications can provide personalized exercise routines, diet plans, and lifestyle suggestions tailored to the predicted age, enhancing health management and engagement for users.

CONCLUSION

The AI-powered age detection project successfully demonstrates how Machine learning and computer vision can estimate human age from facial images with accuracy and efficiency. By integrating the model with cloud APIs and Salesforce, the system showcases practical real-world applications in healthcare, retail, and security. The project highlights the importance of diverse datasets, model optimization, and secure data handling, while future enhancements like real-time detection, privacy protection, and explainable AI can make it even more robust and versatile.